Basic properties I

- A *graph* G (also called a *simple graph*) is composed of two types of objects. It has a finite set

$$V = \{a, b, c \ldots\}$$

of elements called *vertices* (sometimes also called *nodes*) and a set E of pairs of distinct vertices called *edges*. We denote the graph whose vertex set is V and whose edge set is E by

$$G = (V, E).$$

The number n of vertices in the set V is called the *order* of the graph G. If

$$\alpha = \{x, y\} = \{y, x\}$$

is an edge of G, then we say that α *joins* x and y, and that x and y are *adjacent*; we also say that x and α are *incident*, and y and α are *incident*. We also call x and y the *vertices of the edge* α. We take one distinct point, a



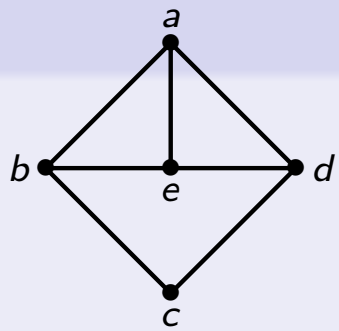
Basic properties II

vertex-point, for each vertex x (labeling the vertex-point with the vertex) and connect two vertex-points by a simple curve if and only if the corresponding vertices determine an edge α of G. We call such a curve an *edge-curve* and label it with α.

Example

A graph $G = (V, E)$ of order 5

$$V = \{a, b, c, d, e\},$$
$$E = \{\{a, b\}, \{b, c\}, \{c, d\}, \{d, a\}, \{e, a\}, \{e, b\}, \{e, d\}\}.$$

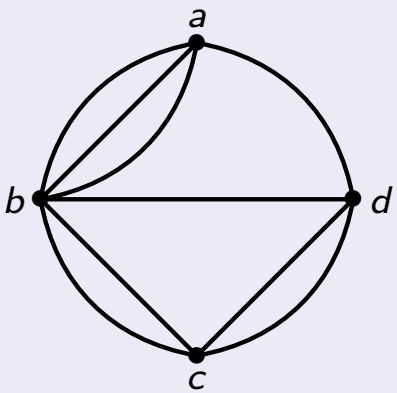


- If we alter the definition of a graph to allow a pair of vertices to form more than one edge, then the resulting structure is called a *multigraph*.
- In a multigraph $G = (V, E)$, E is a multiset. The *multiplicity* of *an edge* $\alpha = \{x, y\}$ is the number of times, $m\{x, y\}$, it occurs in E.

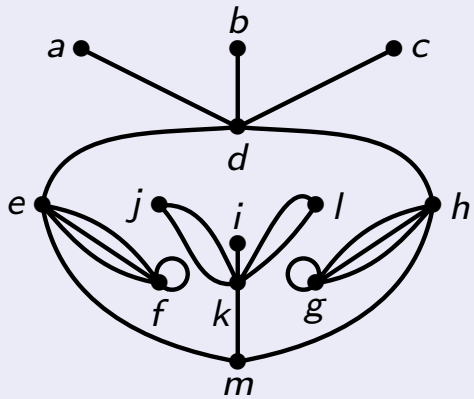
Basic properties III

- The further generalization by allowing *loops*, edges of the form $\{x, x\}$ making a vertex adjacent to itself, is called a general graph.

Example



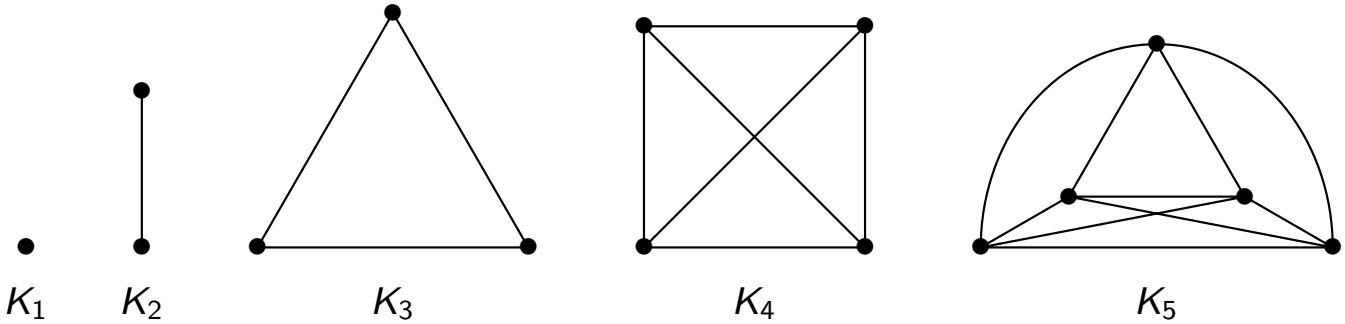
(a) A multigraph of order 4 with 9 edges



(b) A general graph of order 13 with 21 edges (*GraphBuster*)

- A graph of order n is called *complete*, provided that each pair of distinct vertices forms an edge. A complete graph of order n has $n(n-1)/2$ edges and is denoted by K_n.

Basic properties IV



- A general graph G is called *planar*, provided that it can be represented by a drawing in the plane in the manner just described in such a way that two edge-curves intersect only at vertex-points. such a drawing is called a *plane-graph* and is a *planar representation* of G.



Basic properties V

- The *degree* (*valence*) of a vertex x in a general graph G is the number of edges, $\deg(x)$, that are incident with x. If $\alpha = \{x, x\}$ is a loop joining x to itself, then α contributes 2 to the degree of x. To each general graph G, we associate a sequence of numbers that is the list of the degrees of the graph's vertices in nonincreasing order,

$$(d_1, d_2, \cdots, d_n), \quad d_1 \geq d_2 \geq \cdots \geq d_n \geq 0.$$

We call this sequence the *degree sequence* of G.

Theorem

Let G be a general graph. The sum

$$d_1 + d_2 + \cdots + d_n$$

of the degrees of all the vertices of G is an even number, and consequently, the number of vertices of G with odd degree is even.



Basic properties VI

Example

At a party a lot of handshaking takes place between the guests. Show that at the end of the party the number of guests who have shaken hands an odd number of times is even.

- Two general graph $G = (V, E)$ and $G' = (V', E')$ are called *isomorphic*, provided that there is a one-to-one correspondence

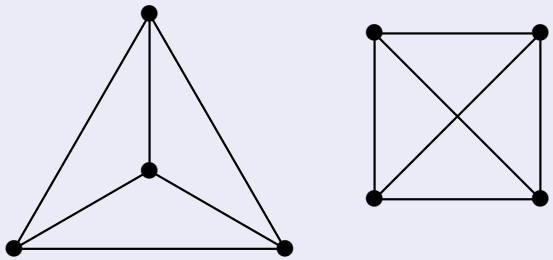
$$\theta : V \to V'$$

between their vertex sets such that, for each pair of vertices x and y of V, there are as many edges of G joining x and y as there are edges of G' joining $\theta(x)$ and $\theta(y)$. The one-to-one correspondence θ is called an *isomorphism* of G and G'.

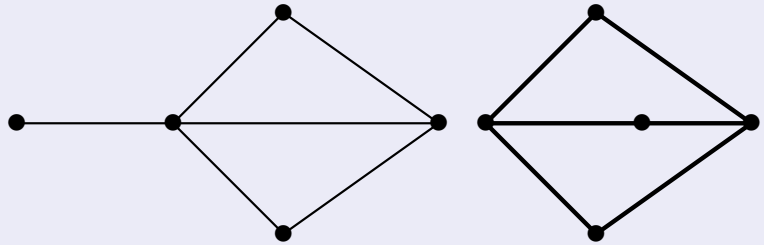


Basic properties VII

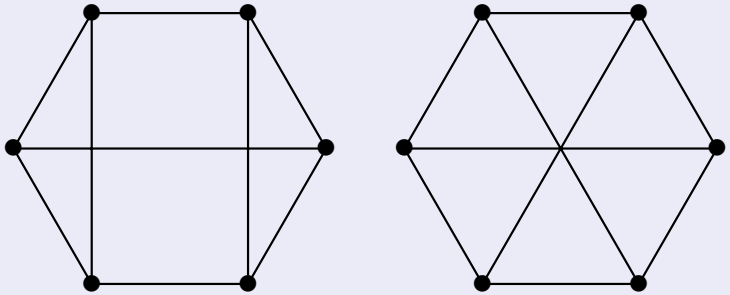
Example



(a)



(b)



(c)

(a) isomorphic graphs; (b) and (c) graphs which are not isomorphic.

Basic properties VIII

Theorem

Two isomorphic general graphs have the same degree sequence, but two graphs with the same degree sequence need not be isomorphic.

- Let $G = (V, E)$ be a general graph. A sequence of m edges of the form

$$\{x_0, x_1\}, \{x_1, x_2\}, \cdots, \{x_{m-1}, x_m\} \tag{11.1}$$

is called a *walk of length* m, and this walk *joins the vertices* x_0 and x_m. We also denote the walk (11.1) by

$$x_0 - x_1 - x_2 - \cdots - x_m.$$

- The walk (11.1) is *closed* or *open* depending on whether $x_0 = x_m$ or $x_0 \neq x_m$.
- If a walk has distinct edges, then it is called a *trail*.
- If, in addition, a walk has distinct vertices (except, possibly, $x_0 = x_m$), then the walk is called a *path*.



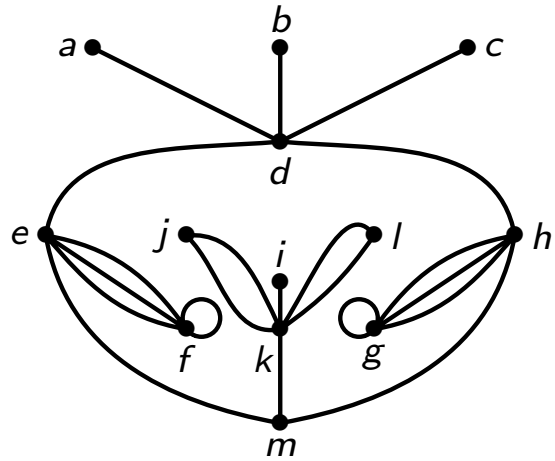
Basic properties IX

- A closed path is called a *cycle*.

Example

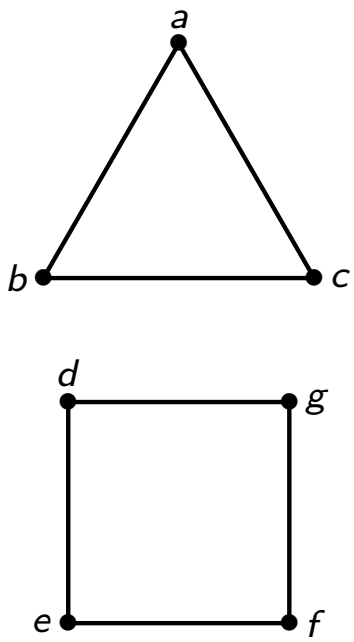
Consider the general graph GraphBuster, we have the following.

1. $a-d-b-d-c-d-h-g-h-m-k-i$ is a walk of length 11 joining vertex a and vertex i, but it is not a trail.
2. $a-d-e-f-e-m-k-l-k-i$ is a trail of length 9 joining a and i but it is not a path.
3. $a-d-e-m-k-i$ is a path of length 5 joining a and i.
4. $d-e-f-e-m-h-d$ is a closed trail of length 6, but it is not a cycle.
5. Each of $f-f$, $e-f-e$, and $d-e-m-h-d$ is a cycle.



Basic properties X

- A general graph G is called *connected*, provided that, for each pair of vertices x and y, there is a walk joining x and y (equivalently, a path joining x and y). Otherwise, G is *disconnected*.
- Let $G = (V, E)$ be a general graph. Let U be a subset of V and let F be a submultiset of E, such that the vertices of each edge in F belong to U. Then $G' = (U, F)$ is also a general graph called a *general subgraph* of G.
- If F consists of all edges of G that join vertices in U, then G' is called an *induced* general subgraph of G and is denoted by G_U.
- In case U is the entire set V of vertices of G then G' is called *spanning*.



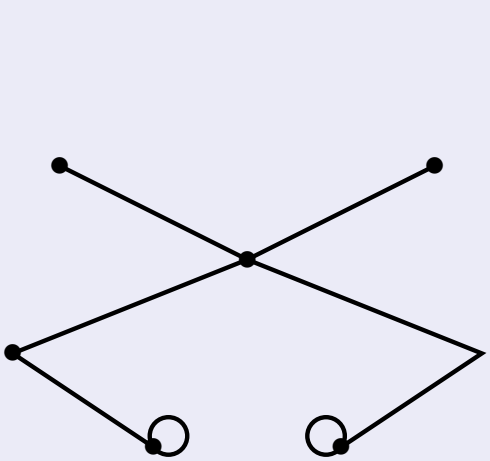
A disconnected graph



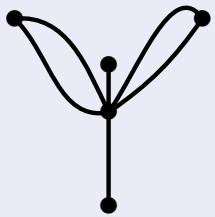
Basic properties XI

Example

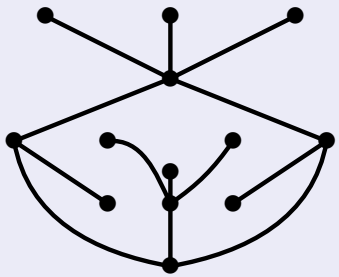
Let G be the general graph GraphBuster,



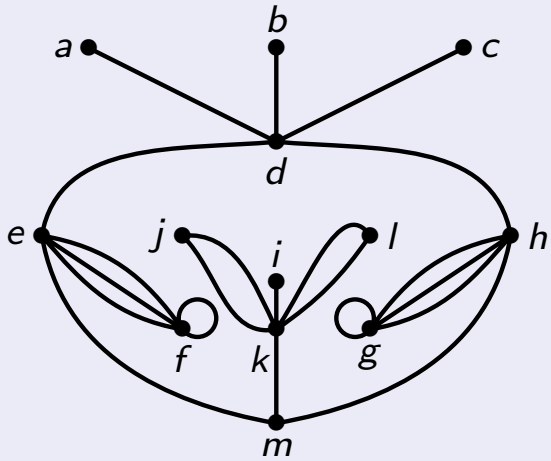
(i)



(ii)



(iii)



(i) A general subgraph which is neither induced nor spanning.

(ii) A general subgraph which is induced but not spanning.

(iii) A general subgraph (which happens to be a graph) which is spanning but not induced.



Basic properties XII

Theorem

Let $G = (V, E)$ be a general graph. Then the vertex set V can be uniquely partitioned into nonempty parts $V_1, V_2, \ldots, V_k$ so that the following conditions are satisfied.

1. The general subgraphs $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$, $\ldots$, $G_k = (V_k, E_k)$ induced by $V_1, V_2, \ldots, V_k$, respectively, are connected.
2. For each $i \neq j$ and each pair of vertices x in V_i and y in V_j, there is no walk that joins x and y.



Basic properties XIII

- The general graphs $G_1, G_2, \ldots, G_k$ in the theorem are the *connected components* of G, which are *maximal* connected induced general subgraphs.

Theorem

Let G and G' be two general graphs. Then the following are necessary conditions for G and G' to be isomorphic.

1. If G is a graph, so is G'.
2. If G is connected, so is G'. More generally, G and G' have the same number of connected components.
3. If G has a cycle of length equal to some integer k, then so does G'.
4. If G has an (induced) general subgraph that is a complete graph K_m of order m, so does G'.

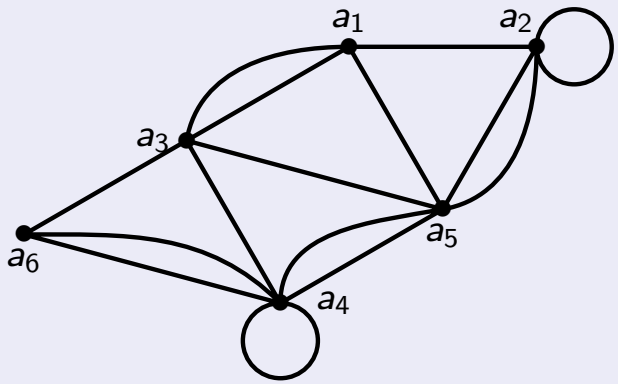


Basic properties XIV

- Let G be a general graph of order n and let its vertices be, in some order, $a_1, a_2, \ldots, a_n$. Let A be the n-by-n array such that the entry a_{ij} in row i, column j equals the number of edges joining the vertices a_i and a_j, $(1 \leq i, j \leq n)$. We always have $a_{ij} = a_{ji}$, and a_{ii} counts the number of loops at vertex a_i. The matrix A is called the *adjacency matrix* of G.
- In case of G is a graph, then A is a matrix of 0's and 1's and the entry a_{ij} equals 1 if and only if a_i and a_j are adjacent in G.

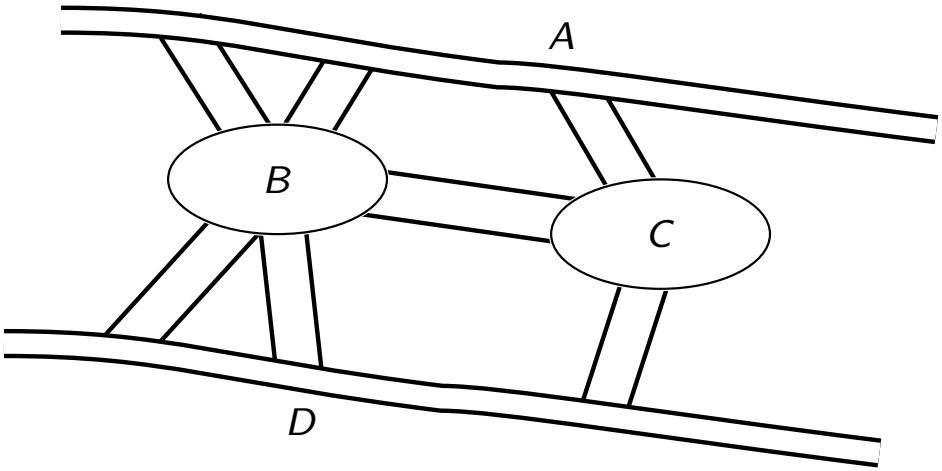
Example

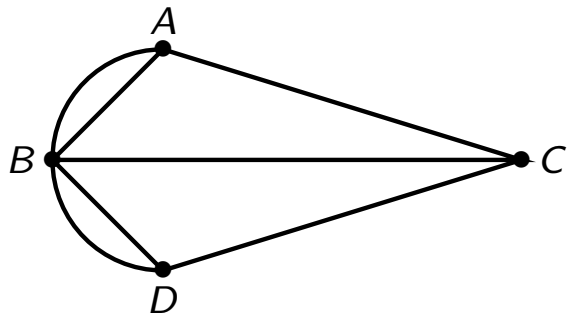
$$\begin{bmatrix} 0 & 1 & 2 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 2 & 0 \\ 2 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 2 & 2 \\ 1 & 2 & 1 & 2 & 0 & 0 \\ 0 & 0 & 1 & 2 & 0 & 0 \end{bmatrix}$$



Eulerian trails I

- The old city of Königsberg in East Prussia was located along the banks and on two islands of the Pregel River, with the four parts of the city connected by seven bridges as shown in Figure.





On Sundays, the citizens of Königsberg would promenade about town, and the problem arose as to whether it was possible to plan a promenade in such a way that each bridge is crossed once and only once.

Eulerian trails II

- A trail in a general graph G is called *Eulerian*, provided that it contains every edge of G.

Lemma

Let $G = (V, E)$ be a general graph and assume that the degree of each vertex is even. Then each edge of G belongs to a closed trail and hence to a cycle.

Algorithm for a closed trail

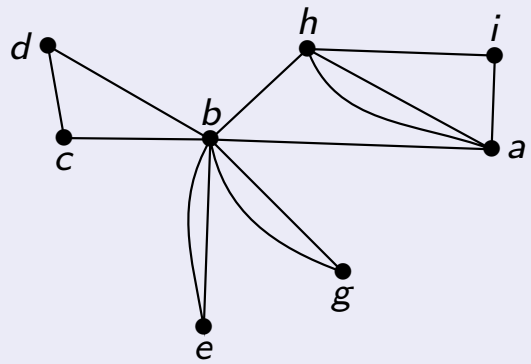
1. Put $i = 1$.
2. Put $W = \{x_0, x_1\}$.
3. Put $F = \{\alpha_1\}$.
4. While $x_i \neq x_0$, do the following.
   i. Locate an edge $\alpha_{i+1} = \{x_i, x_{i+1}\}$ not in F.
   ii. Put x_{i+1} in W (x_{i+1} may already be in W).
   iii. Put α_{i+1} in F.
   iv. Increase i by 1.

Eulerian trails III

Example

By the above algorithm for a closed trail,

i	x_i	α_i	W	F
1	b	$\{a, b\}$	a, b	$\{a, b\}$
2	c	$\{b, c\}$	a, b, c	$\{a, b\}, \{b, c\}$
3	d	$\{c, d\}$	a, b, c, d	$\{a, b\}, \{b, c\}, \{c, d\}$
4	b	$\{d, b\}$	a, b, c, d	$\{a, b\}, \{b, c\}, \{c, d\}, \{d, b\}$
5	h	$\{b, h\}$	a, b, c, d, h	$\{a, b\}, \{b, c\}, \{c, d\}, \{d, b\}, \{b, h\}$
6	a	$\{h, a\}$	a, b, c, d, h	$\{a, b\}, \{b, c\}, \{c, d\}, \{d, b\}, \{b, h\}, \{h, a\}$



We thus obtain a closed trail

$$\{a, b\}, \{b, c\}, \{c, d\}, \{d, b\}, \{b, h\}, \{h, a\}$$

and the cycle

$$\{a, b\}, \{b, h\}, \{h, a\}$$

Eulerian trails IV

containing the edge $\{a, b\}$.

Theorem

Let G be a connected general graph. Then G has a closed Eulerian trail if and only if the degree of each vertex is even.



Eulerian trails V

Example

$$\begin{aligned}
\gamma_1 &= a-b-c-d-b-h-a,\\
\gamma_2 &= b-e-b,\\
\gamma_1 \overset{b}{*} \gamma_2 &= a-b-e-b-c-d-b-h-a,\\
\gamma_3 &= b-g-b,\\
\gamma_1 \overset{b}{*} \gamma_2 \overset{b}{*} \gamma_3 &= a-b-g-b-e-b-c-d-b-h-a,\\
\gamma_4 &= h-i-a-h,\\
\gamma_1 \overset{b}{*} \gamma_2 \overset{b}{*} \gamma_3 \overset{b}{*} \gamma_4 &= a-b-g-b-e-b-c-d-b-h-i-a-h-a.
\end{aligned}$$



Eulerian trails VI

Theorem

Let G be a connected general graph. Then G has a open Eulerian trail if and only if there are exactly two vertices u and v of odd degree. Every open Eulerian trail in G joins u and v.

Theorem

Let G be a connected general graph and suppose that the number of vertices of G with odd degree is $m > 0$. Then the edges of G can be partitioned into $m/2$ open trails. It is impossible to partition the edges of G into fewer than $m/2$ open trails.

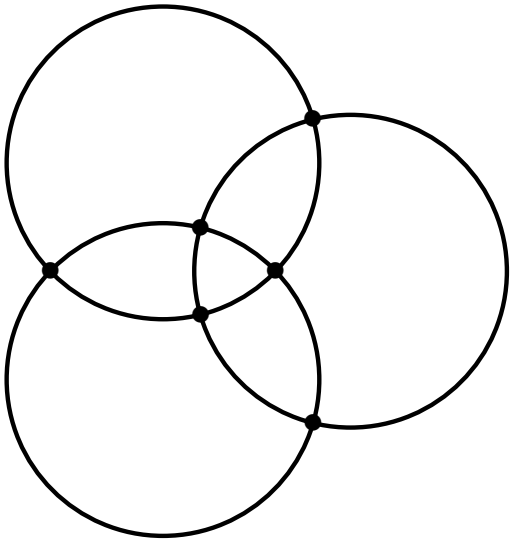
Example

(a) All of the vertices in the graph have degree equal to 4 and the graph is traceable.

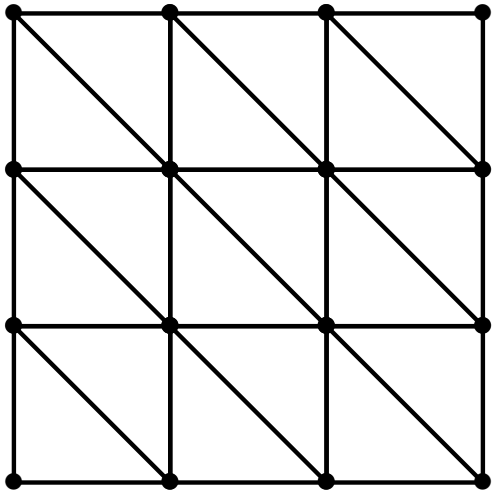
(b) The graph has 2 vertices of odd degree and hence it has an open Eulerian trail joining the 2 vertices of odd degree.

(c) The graph has 4 vertices of odd degree and is not traceable.

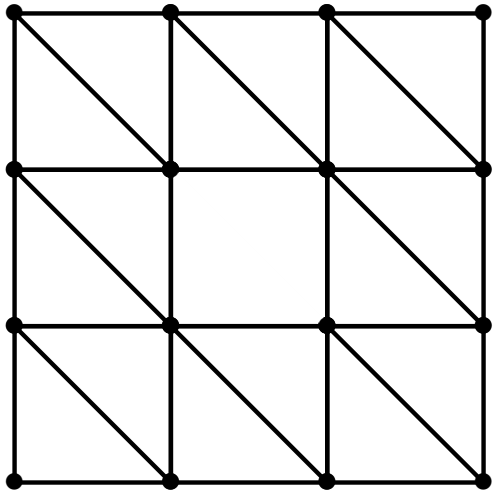
Eulerian trails VII



(a)



(b)



(c)

Chinese postman problem

Let G be a connected general graph. Find a closed walk of shortest length which uses each edge of G at least once.

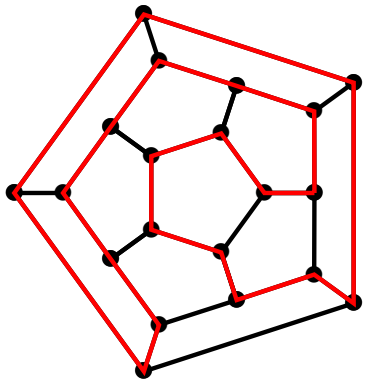
Eulerian trails VIII

Theorem

Let G be a connected general graph having K edges. Then there is a closed walk in G of length $2K$ in which the number of times an edge is used twice its multiplicity.



Hamilton paths and cycles I



- Hamilton's puzzle can be formulated for any graph

  Given a graph G can one determine a route along the edges of G that begins at some vertex and then returns there after having visited every other vertex exactly once?

- A solution to Hamilton's puzzle for a graph G is called a Hamilton cycle. More precisely, a *Hamilton cycle* of a graph G of order n is a cycle of G of length n. Hence, a Hamilton cycle in the graph G of order n is a cycle

  $$x_1 - x_2 - \cdots - x_n - x_1$$

  of length n where $x_1, x_2, \ldots, x_n$ are the n vertices of G in some order. A *Hamilton path* in G joining vertices a and b is a path

  $$a = x_1 - x_2 - \cdots - x_n = b$$

  of length $n-1$ of G.



Hamilton paths and cycles II

Example

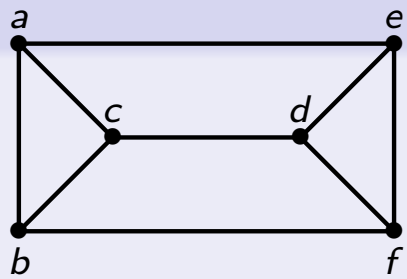
A complete graph K_n of order $n \geq 3$ has a Hamilton cycle. K_n has $n!$ Hamilton paths and $(n-1)!$ Hamilton cycles.

Example

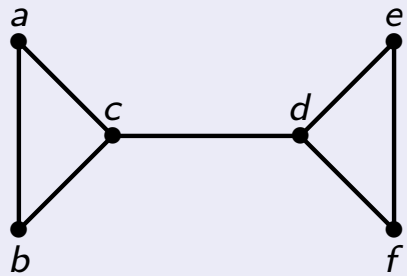
Hamilton cycles

(a) $a-b-c-d-f-e-a$ is a Hamilton cycle, thus $a-b-c-d-f-e$ is a Hamilton path. Another Hamilton path is $a-b-c-d-e-f$, but this Hamilton path cannot be extended to a Hamilton cycle.

(b) A Hamilton path is $a-b-c-d-e-f$, but this graph does not have a Hamilton cycle.



(a)



(b)

Hamilton paths and cycles III

- An edge of a connected graph is called a *bridge*, provided its removal from the graph leaves a disconnected graph.

Theorem

A connected graph with a bridge does not have a Hamilton cycle.

- Let G be a graph of order n and consider the following property which may or may not be satisfied in G

Definition

Ore property – For all pairs of distinct vertices x and y that are not adjacent,

$$\deg(x) + \deg(y) \geq n.$$



Hamilton paths and cycles IV

Theorem

Let G be a graph of order $n \geq 3$ that satisfies the Ore property. Then G has a Hamilton cycle.

Algorithm for a Hamilton cycle

(1) Start with any vertex and, by attaching adjacent vertices at either end, construct a longer and longer path until it is not possible to make it any longer. Let the path be

$$\gamma : y_1 - y_2 - \cdots - y_m. \tag{11.2}$$

(2) Check to see if y_1 and y_m are adjacent.
   (i) If y_1 and y_m are not adjacent, go to (3). Else go to (ii).
   (ii) If $m = n$, stop and output the Hamilton cycle $y_1 - y_2 - \cdots - y_m - y_1$. Else go to (iii).



Hamilton paths and cycles V

(iii) Locate a vertex z not on γ and a vertex y_k on γ such that z is adjacent to y_k. Replace γ with the path of length $m+1$ given by

$$z - y_k - \cdots - y_m - y_1 - \cdots - y_{k-1},$$

and go back to (2).

(3) Locate a vertex y_k with $1 < k < m$ such that y_1 and y_k are adjacent and y_{k-1} and y_m are adjacent. Replace γ with the path

$$y_1 - \cdots - y_{k-1} - y_m - \cdots - y_k.$$

The two ends of this path, namely, y_1 and y_k are adjacent, and go back to (2)(ii).



Hamilton paths and cycles VI

Corollary

A graph of order $n \geq 3$, in which each vertex has degree at least $n/2$, has a Hamilton cycle.

Theorem

A graph of order n, in which the sum of the degrees of each pair of nonadjacent vertices is at least $n-1$, has a Hamilton path.

Example

The traveling salesperson problem – Consider a salesperson who is planning a business trip that takes him to certain cities in which he has customers and then brings him back home to the city from whence he started. Between some of the pairs of cities he has to visit there is direct air service; between others there is not. Can he plan the trip so that he flies into each city to be visited exactly once?

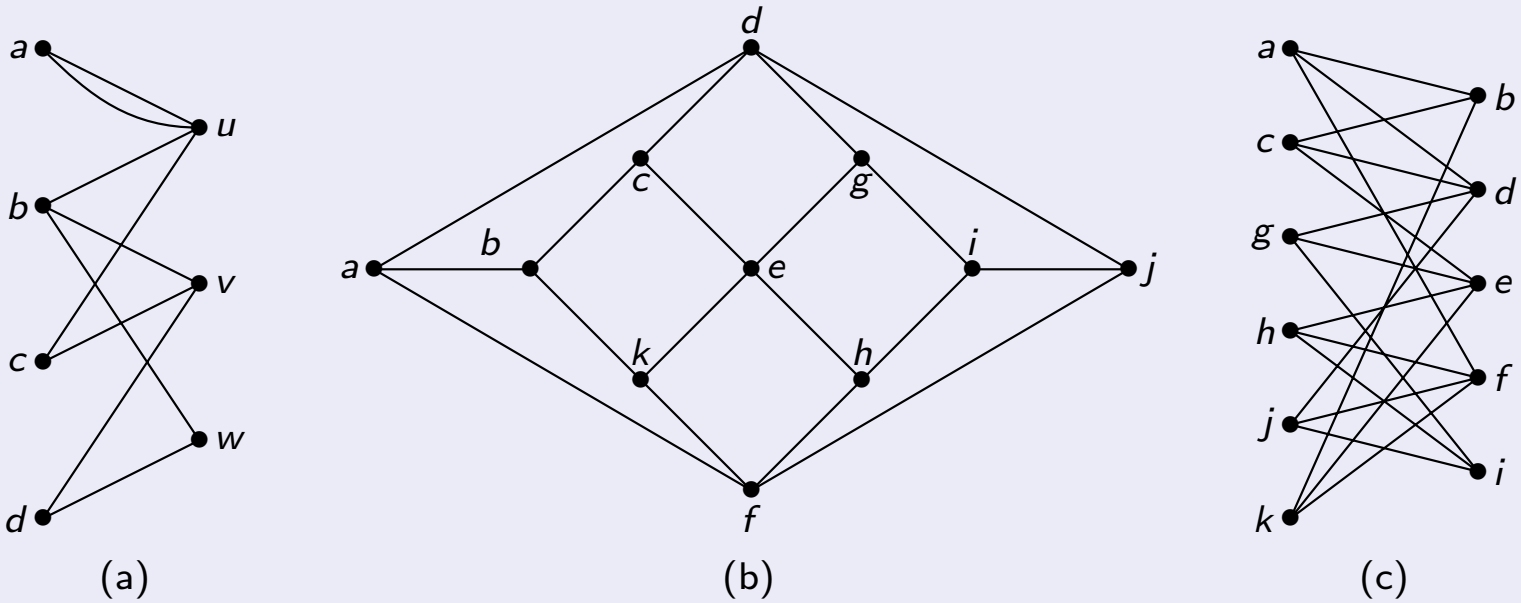


Bipartite multigraphs I

- Let $G = (V, E)$ be a multigraph. Then G is called *bipartite*, provided that the vertex set V may be partitioned into two subsets X and Y so that each edge of G has one vertex in X and one vertex in Y. A pair X, Y with this property is called a *bipartition* of G (and of its vertex set V).

Example

(b) and (c) are isomorphic.



Bipartite multigraphs II

- A multigraph that is isomorphic to a bipartite multigraph is also bipartite.

Example

Let G be the graph whose vertices are the integers from 1 to 20, with two integers joined by an edge if and only if their difference is an odd integer. The vertices of G are naturally partitioned into the even integers and the odd integers. Thus, G is a bipartite graph with bipartition $X = \{1, 3, \cdots, 17, 19\}$, $Y = \{2, 4, \cdots, 18, 20\}$.

- A bipartite graph G with bipartition X, Y is called *complete*, provided that each vertex in X is adjacent to each vertex in Y. A complete bipartite graph with m left vertices and n right vertices is denoted by $K_{m,n}$.

Theorem

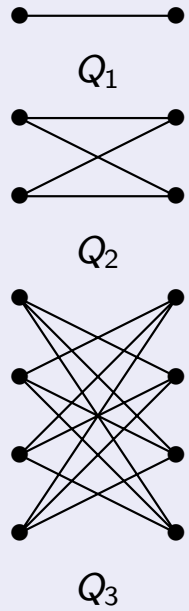
A multigraph is bipartite if and only if each of its cycles has even length.



Bipartite multigraphs III

Example

Let n be a positive integer. We consider the set of all n-tuples of 0's and 1's as the vertices of a graph Q_n with two vertices joined by an edge if and only if they differ in exactly one coordinate. If $x = (x_1, \cdots, x_n)$ and $y = (y_1, \cdots, y_n)$ are joined by an edge, then the number of 1's in y is either 1 more or 1 less than the number of 1's in x. Let X and Y consist of those n-tuples that have an even and odd number of 1's, respectively. Then two distinct vertices in X differ in at least two coordinates and hence are not adjacent. Similarly, two distinct vertices in Y are not adjacent. Hence, Q_n is the graph of vertices and edges of an n-dimensional cube.



Q_n is a bipartite graph with bipartition X, Y.

Bipartite multigraphs IV

Theorem

Let G be a bipartite graph with bipartition X, Y. If $|X| \neq |Y|$, then G does not have a Hamilton cycle. If $|X| = |Y|$, then G does not have a Hamilton path that begins at a vertex in X and ends at a vertex in X. If X and Y differ by at least 2, then G does not have a Hamilton path. If $|X| = |Y| + 1$, then G does not have a Hamilton path that begins at X and ends at Y, or vice versa.

Example

The Knight's Tour Problem

58	43	60	37	52	41	62	35
49	46	57	42	61	36	53	40
44	59	48	51	38	55	34	63
47	50	45	56	33	64	39	54
22	7	32	1	24	13	18	15
31	2	23	6	19	16	27	12
8	21	4	29	10	25	14	17
3	30	9	20	5	28	11	26

1	4	7	10
12	9	2	5
3	6	11	8



Trees I

Suppose we want to build a connected graph of order n, using the smallest number of edges that we can "get away with". One simple method of construction is to select one vertex and join it by an edge to each of the other $n-1$ vertices. The result is a complete bipartite graph $K_{1,n-1}$, called a *star*.

Theorem

A connected graph of order n has at least $n-1$ edges. Moreover, for each positive integer n, there exist connected graphs with exactly $n-1$ edges. Removing any edge from a connected graph of order n with $n-1$ edges leaves a disconnected graph, and hence each edge is a bridge.

- A *tree* is defined to be a connected graph that becomes disconnected upon the removal of any edge.



Trees II

- A tree is a connected graph, each of whose edge is a bridge.

Theorem

A connected graph of order $n \geq 1$ is a tree if and only if it has exactly $n-1$ edges.

Lemma

Let G be a connected graph and let $\alpha = \{x, y\}$ be an edge of G. Then α is a bridge if and only if no cycle of G contains α.

Theorem

Let G be a connected graph of order n. Then G is a tree if and only if there are no cycle in G.

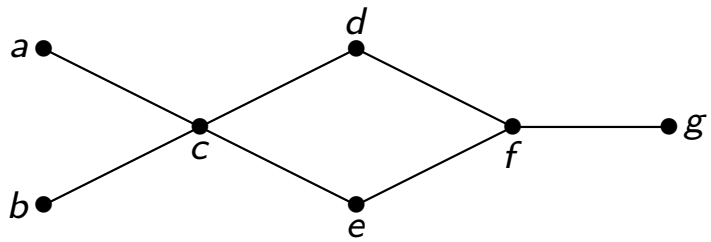


Trees III

Theorem

A graph is a tree if and only if every pair of distinct vertices x and y is joined by a unique path. This path is necessarily a shortest joining x and y, that is, a path of length $d(x, y)$.

- Let G be a graph. A *pendent vertex* of G is a vertex whose degree is equal to 1. Thus, a pendent vertex is incident with exactly one edge, and any edge incident with a pendent vertex is called a *pendent edge*.



Trees IV

Theorem

Let G be a tree of order $n \geq 2$. Then G has at least 2 pendent vertices.

Example

What is the smallest and largest number of pendent vertices a tree G of order $n \geq 2$ can have?

Solution

The number of pendent vertices is 2 when $n = 2$. The smallest and largest number of pendent vertices of a tree G of order n is 2 and $n - 1$ when $n \geq 3$.

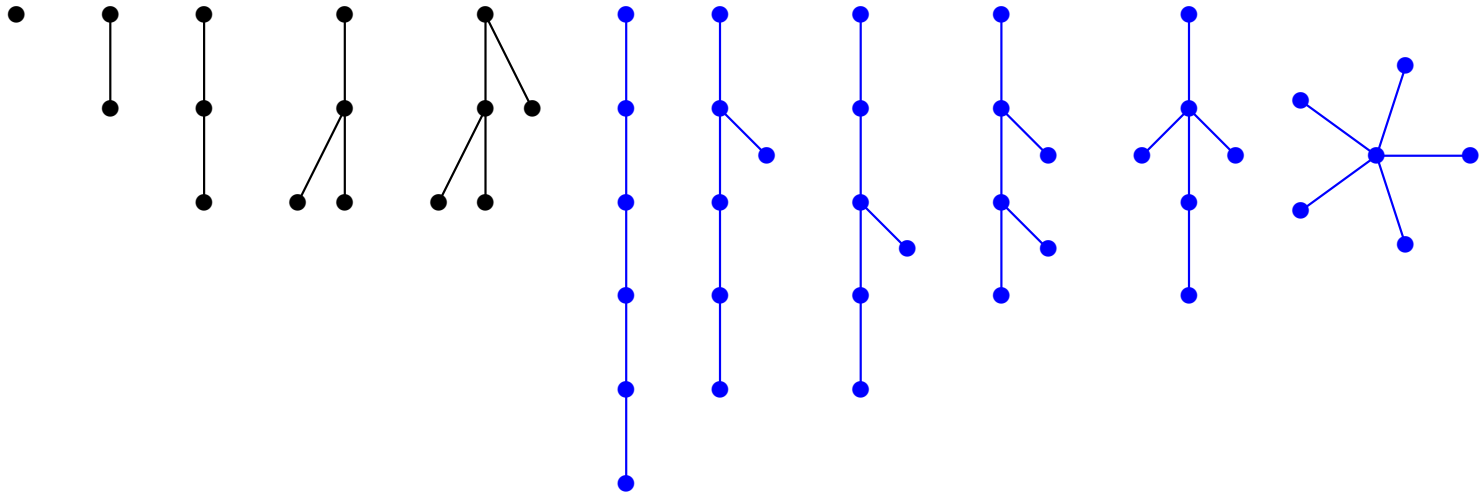
Example

How to grow trees?



Trees V

- The number t_n of nonisomorphic trees of order n satisfies $t_1 = 1$, $t_2 = 1$, $t_3 = 1$, $t_4 = 2$, $t_5 = 3$, and $t_6 = 6$.



- A tree that is a spanning subgraph of a graph G is called a *spanning tree* of G.



Trees VI

Theorem

Every connected graph has a spanning tree.

Algorithm for a spanning tree

Let $G = (V, E)$ be a connected graph of order n.

i. Set F equal to E.

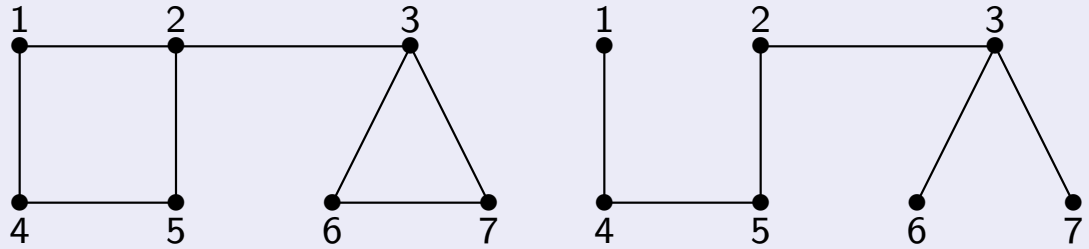
ii. While there is an edge α of F such that α is not a bridge of the graph $T = (V, F)$, remove α from F.

The terminal graph $T = (V, F)$ is a spanning tree of G.



Trees VII

Example



Theorem

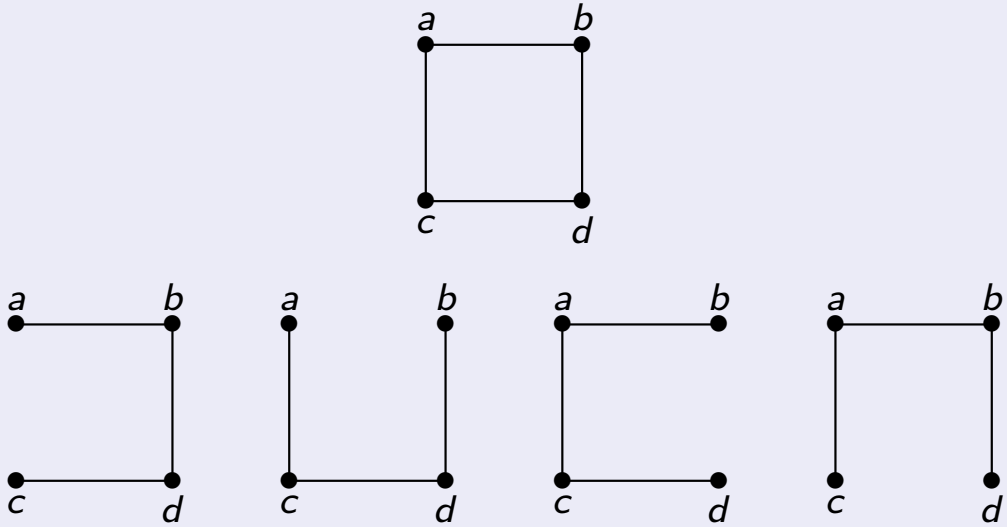
Let T be a spanning tree of a connected graph. Let $\alpha = \{a, b\}$ be an edge of G that is not an edge of T. Then there is an edge β of T such that the graph T' obtained from T by inserting α and deleting β is also a spanning tree of G.

Theorem

Let T_1 and T_2 be spanning trees of a connected graph G. Let β be an edge of T_1. Then there is an edge α of T_2 such that the graph obtained from T_1 by inserting α and deleting β is a spanning tree of G.

Trees VIII

Example



The Shannon switching game I

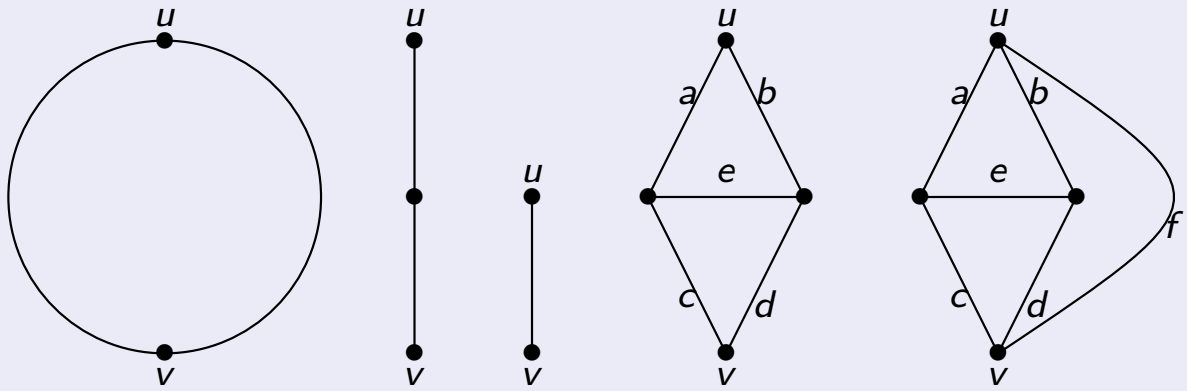
- Shannon's game is played by two people, call here the *positive player* P and the *negative player* N, who alternate turns. Let $G = (V, E)$ be a multigraph in which two of its vertices u and v have been distinguished.
- The goal of the positive player is to construct a path between the distinguished vertices u and v.
- The goal of the negative player is to deny the positive player his goal, that is, to destroy all paths between u and v.
- The play of the game proceeds as follows. When it is N's turn, N destroys some edge of G by putting a negative sign - on it. When it is P's turn, P puts a positive sign $+$ on some edge of G, which now cannot be destroyed by N.
- Play proceeds until one of the players achieves his goal
  1. There is a path between u and v that has only $+$ signs on its edge. In this case, the *positive player has won*.
  2. Every path in G between u and v contains a - sign on at least one of its edges; that is, N has destroyed all paths between u and v. In this case the *negative player has won*.



The Shannon switching game II

- *positive/negative game* – the positive/negative player, whether he plays first or second, has a winning strategy.
- *neutral game*, the player who plays first has a winning strategy.

Example



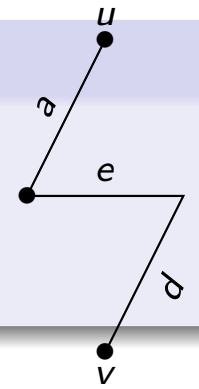
(a) (b) (c) (d) (e)

(a) and (e) are positive games. (b) is a negative game. (c) and (d) are neutral games.

The Shannon switching game III

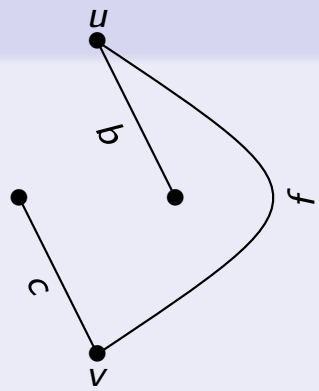
Theorem

A neutral game is converted into a positive game if a new edge of joining the distinguished vertices u and v is added to the multigraph of the game.



Theorem

The game determined by a multigraph $G = (V, E)$ with distinguished vertices u and v is a positive game if and only if there is a subset U containing u and v of the vertex set V such that the induced multigraph G_U has two spanning trees, T_1 and T_2, with no common edges.



Initially, we have the spanning trees T_1 and T_2 of G_U with no edges in common, and we now label these trees as

$$T_1^{(0)} = T_1 \qquad \text{and} \qquad T_2^{(0)} = T_2.$$

The Shannon switching game IV

The first sequence of play

Player N goes first and puts a - on some edge β. We consider two cases.

Case 1: β is an edge of one of the trees $T_1^{(0)}$ and $T_2^{(0)}$, say, the tree $T_1^{(0)}$.

It follows that there is an edge α of $T_2^{(0)}$ such that the graph obtained from $T_1^{(0)}$ by inserting α and deleting β is a spanning tree $T_1^{(1)}$ of G_U. Our instructions to P are to put a $+$ on the edge α. Let $T_2^{(1)} = T_2^{(0)}$. The trees $T_1^{(1)}$ and $T_2^{(1)}$ have exactly one edge in common, namely, the edge α with a $+$ on it.

Case 2: β is neither an edge of $T_1^{(0)}$ nor an edge of $T_2^{(0)}$.

Our instructions to P are now to place a $+$ on any edge α of $T_1^{(0)}$ or of $T_2^{(0)}$, say, an edge α of $T_1^{(0)}$. It follows that there is an edge γ of $T_2^{(0)}$ such that the graph obtained from $T_2^{(0)}$ by inserting α and deleting γ is a spanning tree $T_2^{(1)}$ of G_U. Let $T_1^{(1)} = T_1^{(0)}$. The trees $T_1^{(1)}$ and $T_2^{(1)}$ have only the edge α with a $+$ in common.



The Shannon switching game V

The second sequence of play

Player N puts a - on a second edge δ of G, and we seek a counter-move for P. The determination of an edge ρ on which P should put a $+$ is very much like that in the first sequence of play

Case 1: δ is an edge of one of the trees $T_1^{(1)}$ and $T_2^{(1)}$, say, the tree $T_2^{(1)}$.

There is an edge ρ of $T_1^{(1)}$ such that the graph $T_1^{(2)}$ obtained from $T_1^{(1)}$ by inserting the edge δ and deleting the edge ρ is a spanning tree of G_U. Our instructions to P are to put a $+$ on the edge ρ. We let $T_2^{(2)} = T_2^{(1)}$.

Case 2: δ is neither an edge of $T_1^{(1)}$ nor of $T_2^{(1)}$.

Our instructions to P are to place a $+$ on any available edge α of $T_1^{(1)}$ and $T_2^{(1)}$, say, an edge ρ of $T_1^{(1)}$. There exists an edge ϵ of $T_2^{(1)}$ such that the graph $T_2^{(2)}$ obtained from $T_2^{(1)}$ by inserting the edge ρ and deleting the edge ϵ is a spanning tree of G_U. We let $T_1^{(2)} = T_1^{(1)}$.



The Shannon switching game VI

- Let $G = (V, E)$ be a multigraph with distinguished vertices u and v. Let G^* be the multigraph obtained from G by inserting a new edge joining u and v. Then the following conclusions can be drawn.
  1. The game played with G, u, and v is a neutral game if and only if it is not a positive game, but the game played with G^*, u, and v is a positive game.
  2. The game played with G, u, and v is a negative game if and only if neither the game played with G, u, and v nor the game played with G^*, u, and v are positive games.



More on trees I

Algorithm to grow a spanning tree

Let $G = (V, E)$ be a graph of order n and let u be any vertex.

1. Put $U = \{u\}$ and $F = \emptyset$.
2. While there exists a vertex x in U and a vertex y not in U such that $\alpha = \{x, y\}$ is an edge of G,
   1. Put the vertex y in U.
   2. Put the edge α in F.
3. Put $T = \{U, F\}$.

Theorem

Let $G = (V, E)$ be a graph. Then G is connected if and only if the graph $T = (U, F)$ constructed by carrying out the preceding algorithm is a spanning tree of G.



More on trees II

- We now describe a way to construct a spanning tree called a *breadth-first spanning tree*, abbreviated by *BFS-tress*, rooted at a prescribed vertex u.
- We attach two numbers to each vertex x. One of these is called its *breadth-first number*, denoted by $bf(x)$. The breadth-first numbers represent the order in which vertices are put into the BFS-tree. The other number represents the distance between u and x in the BFS-tree, and is denoted by $D(x)$.



More on trees III

BF-algorithm to grow a BFS-tree rooted at u

Let $G = (V, E)$ be a graph of order n and let u be any vertex.

(1) Put $i = 1$, $U = \{u\}$, $D(u) = 0$, $bf(u) = 1$, $F = \emptyset$, and $T = (U, F)$.

(2) If there is no edge in G that joins a vertex x in U to a vertex y not in U, then stop. Otherwise determine an edge $\alpha = \{x, y\}$ with x in U and y not in U such that x has the smallest breadth-first number $bf(x)$, and do the following:

i. Put $bf(y) = i + 1$.
ii. Put $D(y) = D(x) + 1$.
iii. Put the vertex y into U.
iv. Put the edge α into F.
v. Put $T = \{U, F\}$.
vi. Increase i by 1 and go back to (2).



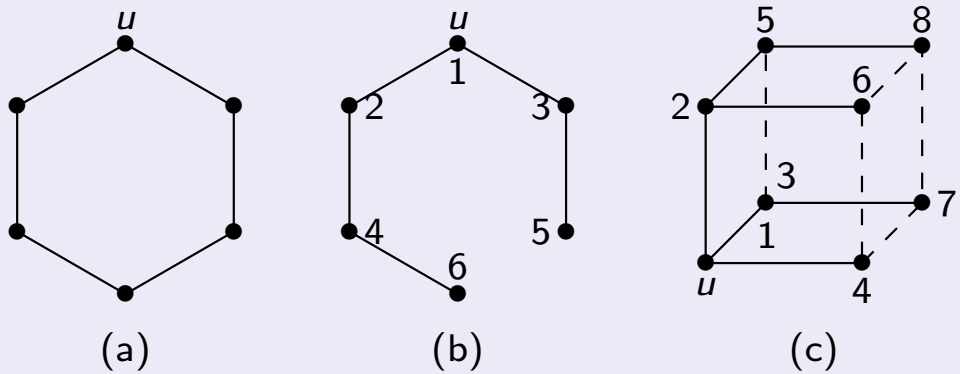
More on trees IV

Theorem

Let $G = (V, E)$ be a graph and let u be any vertex of G. Then G is connected if and only if the graph $T = (U, F)$ constructed by carrying out the preceding BF-algorithm is a spanning tree of G. If G is connected, then, for each vertex y of G, the distance in G between u and y equals $D(y)$, and this is the same as the distance between u and y in T.

Example

Each BFS-tree of a complete graph K_n is a star $K_{1,n}$. A BFS-tree of the cycle of length 6 shown in (a) is the tree shown in (b). A BFS-tree of the graph Q_3 is shown in (c).



(a) (b) (c)

More on trees V

- We now describe a way to carry out the algorithm to grow a tree that produces a spanning tree that is as deep as possible. A spanning tree produced by this algorithm is called a *depth-first spanning tree*, abbreviated a *DFS-tree*, rooted at a vertex u.
- We attach a number to each vertex x, called its *depth-first number*, and denoted by $df(x)$.



More on trees VI

DF-algorithm to grow a DFS-tree rooted at u

Let $G = (V, E)$ be a graph of order n and let u be any vertex.

(1) Put $i = 1$, $U = \{u\}$, $df(u) = 1$, $F = \emptyset$, and $T = (U, F)$.

(2) If there is no edge in G that joins a vertex x in U to a vertex y not in U, then stop. Otherwise determine an edge $\alpha = \{x, y\}$ with x in U and y not in U such that x has the largest depth-first number $df(x)$, and do the following.

i. Put $df(y) = i + 1$.
ii. Put the vertex y into U.
iii. Put the edge α into F.
iv. Put $T = \{U, F\}$.
v. Increase i by 1 and go back to (2).

Theorem

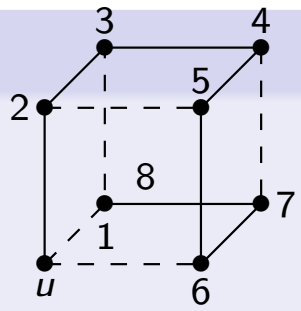
Let $G = (V, E)$ be a graph and let u be any vertex of G. Then G is connected if and only if the graph $T = (U, F)$, constructed by carrying out the preceding DF-algorithm, is a spanning tree of G.



More on trees VII

Example

Each DFS-tree of a complete graph K_n is a path. A DFS-tree of a cycle of any length is also a path. A DFS-tree of the graph Q_3 is shown in the figure.



- Let $G = (V, E)$ be a graph in which to each edge $\alpha = \{x, y\}$ there is associated a nonnegative number $c(\alpha) = c\{x, y\}$, called its *weight*. We call G a *weighted graph* with weight function c. The *weight of a walk*

$$\gamma : \{x_0, x_1\}, \{x_1, x_2\}, \cdots, \{x_{k-1}, x_k\}$$

in G is defined to be

$$c(\gamma) = c\{x_0, x_1\} + c\{x_1, x_2\} + \cdots + c\{x_{k-1}, x_k\},$$

the sum of the weights of the edges of γ. The *weighted-distance* $d_c(x, y)$ between a pair of vertices x and y of G is the smallest weight of all the walks joining x and y. Starting with a vertex u in a connected graph G, we show



More on trees VIII

how to compute $d_c(u, x)$ for each vertex x and construct a spanning tree rooted at u such that the weighted-distance between u and each vertex x equals $d_c(u, x)$. We call such a spanning tree a *distance-tree for* u.

Algorithm to a distance-tree for u (Dijkstra's Algorithm)

Let $G = (V, E)$ be a weighted graph of order n and let u be any vertex.

(1) Put $U = \{u\}$, $D(u) = 0$, $F = \emptyset$, and $T = (U, F)$.

(2) If there is no edge in G that joins a vertex x in U to a vertex y not in U, then stop. Otherwise determine an edge $\alpha = \{x, y\}$ with x in U and y not in U such that $D(x) + c\{x, y\}$ is as small as possible, and

i. Put the vertex y into U.
ii. Put the edge α into F.
iii. Put $D(y) = D(x) + c\{x, y\}$ and go back to (2).



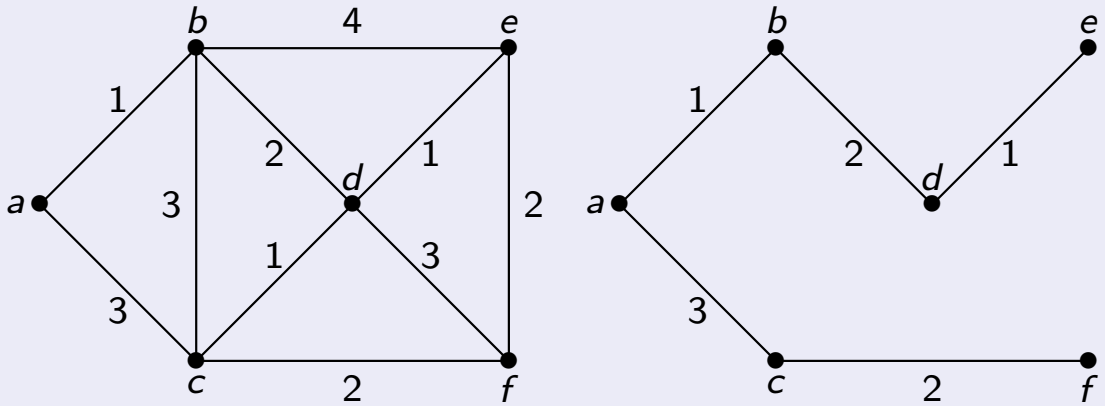
More on trees IX

Theorem

Let $G = (V, E)$ be a weighted graph and let u be any vertex of G. Then G is connected if and only if the graph $T = (U, F)$ obtained by carrying out the preceding algorithm is a spanning tree of G. If G is connected, then, for each vertex y of G, the distance in G between u and y equals $D(y)$, and this is the same as the distance between u and y in the weighted tree T.

Example

A distance-tree rooted at a.



More on trees X

- Let $G = (V, E)$ be a weighted connected graph with weight function c. We define the *weight of a subgraph H* of G to be

$$c(H) = \sum_{\{\alpha \text{ an edge of } H\}} c(\alpha),$$

the sum of the weights of the edges of H. A spanning tree of G that has the smallest weight of all spanning trees of G is a *minimum-weight spanning tree*. We now describe an algorithm known as Kruskal's algorithm. This algorithm is also known as a *greedy algorithm*.

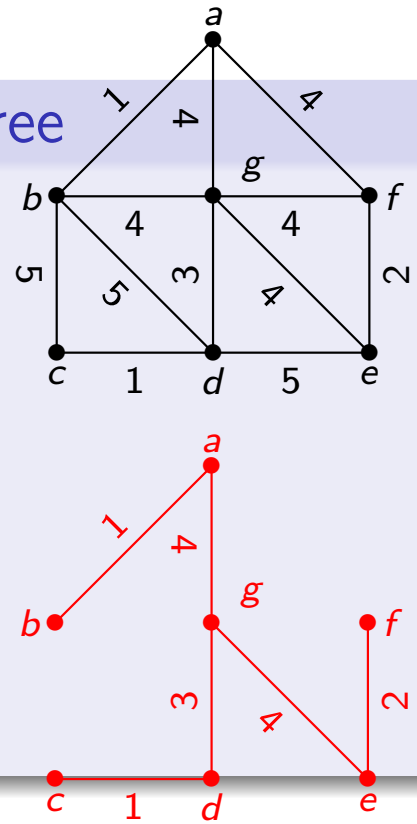


More on trees XI

Greedy algorithm for a minimum-weight spanning tree

Let $G = (V, E)$ be a weighted connected graph with weight function c.

1. Put $F = \emptyset$.
2. While there exists an edge α not in F such that $F \cup \{\alpha\}$ does not contain the edges of a cycle of G, determine such an edge α of minimum weight and put α in F.
3. Put $T = (V, F)$.



Theorem

Let $G = (V, E)$ be a weighted connected graph with weight function c. Then the preceding greedy algorithm constructs a minimum-weight spanning tree $T = (V, F)$ of G.

More on trees XII

Prim's algorithm for a minimum-weight spanning tree

Let $G = (V, E)$ be a weighted connected graph with weight function c and let u be any vertex of G.

1. Put $i = 1$, $U_1 = \{u\}$, $F_1 = \emptyset$ and $T_1 = (U_1, F_1)$.
2. For $i = 1, 2, \ldots, n-1$, do the following:
   1. Locate an edge $\alpha_i = \{x, y\}$ of smallest weight such that x is in U_i and y is not in U_i.
   2. Put $U_{i+1} = U_i \cup \{y\}$, $F_{i+1} = F_i \cup \{\alpha_i$ and $T_{i+1} = (U_{i+1}, F_{i+1})$.
   3. Increase i to $i + 1$.
3. Output $T_{n-1} = (U_{n-1}, F_{n-1})$. (Here $U_{n-1} = V$.)

Theorem

Let $G = (V, E)$ be a weighted graph with weight function c. Then Prim's algorithm constructs a minimum-weight spanning tree $T = (V, F)$ of G.



More on trees XIII

Example

